COLEÇÃO TÉCNICA INTUIÇÃO

Luigi Scofano de Araujo

QA e manutenção

Testes regressão migrações e limites de Intuição

Este manual reúne procedimentos para investigar falhas, manter o conteúdo e proteger progresso em Intuição. A conclusão principal é dupla: as 640 soluções de referência atuais foram reproduzidas com sucesso nos motores; a suíte global ampla do baseline não está verde. Esses resultados têm escopos diferentes e precisam continuar separados.

Baseline 2354a3c. A execução ampla registrada em 1 nov 2026 reportou 701 testes, 521 aprovados e 180 falhas, em aproximadamente 31,8 segundos. A contagem histórica 629/469/160 pertence a 4c70d69 e não é o resultado deste baseline.

Mapa de leitura

Seção	Uso
2 Diagnóstico da base	Entender os resultados sem esconder falhas
3 Comandos de trabalho	Executar testes e distinguir scripts mutáveis
4 Camadas de teste	Escolher a evidência que responde à pergunta
5 Matriz de regressão	Cobrir modos, controles e resultados
6 Saves e migrações	Proteger identidades e histórico
7 Assíncrono e geração	Testar cancelamento, worker e fallback
8 Importação e limites	Exercitar entradas inválidas e orçamentos
9 Manutenção	Triar defeitos e revisar mudanças
10 Aceitação	Definir saída, limites e documentação
11 Fechamento local	Resultados finais e limites da revisão técnica

A revisão técnica local foi concluída em f0e7fcf, separada do Site. A suíte atual passou 536/536 testes, com zero arquivos mistos sem classificação. A seção 11 detalha o fechamento. Isso não autoriza publicação nem resolve licenças ou o conteúdo de um export público.

2 Diagnóstico do baseline

Verificação	Resultado	Interpretação
Suíte ampla no baseline	701 totais / 521 passam / 180 falham	Há trabalho de triagem; não declarar aprovação geral
Soluções dos catálogos ativos	640 / 640 reproduzidas	Cada referência resolve no motor atual
Auditoria anterior 4c70d69	629 / 469 / 160	Histórico de outra revisão, não substituir o atual
Revisão local f0e7fcf	536/536 atuais; zero mistos	Classificação encerrada; Site ainda no baseline

Classes de falha já identificadas

• Testes que importam motores de TRAMA ou VÉRTICE removidos do catálogo ativo. Sua falha não prova que um dos 11 jogos atuais está quebrado.

• Expectativas de 500 ou 1.000 fases, ou de sete mapas CERCO, em vez das campanhas de 80 e das 28 variantes atuais.

• Hashes que congelavam uma entrega antiga e agora rejeitam mudanças posteriores legítimas. São provas históricas de escopo, não necessariamente contratos longitudinais.

• Mocks de DOM sem métodos nativos utilizados pela UI, como Element.after; extrações de funções que não incluem auxiliares novos.

O que não pode ser descartado

Falha real confirmada. A normalização de um save histórico de SINTONIA com 500 tentativas arquivou 482 referências na primeira passagem e 964 na segunda, estabilizando na terceira. A causa investigada envolve a revisão de conteúdo usada na chave de arquivo. O caso observado não mudou movimentos nem conclusões, mas viola idempotência. A correção local 2dea81c passou o conjunto focado de 18 testes; não foi publicada e não corrige retroativamente o baseline.

Uma classe explicada não autoriza arquivar todas as falhas semelhantes sem ler a asserção. Migrações, validação de conteúdo, regras e limites de busca merecem triagem individual. Se a fixture mistura versões, crie uma fixture correta e preserve a antiga como evidência histórica. Se há um defeito real, corrija a produção e acrescente um teste que falhe antes da correção.

Testes focados verdes demonstram cobertura útil, mas não anulam falhas globais. O mesmo vale para o QA registrado da revisão SINTONIA v5: seu escopo pode ser forte e ainda assim não cobrir os outros jogos ou todos os testes históricos.

Fontes desta seção: Log amplo do baseline 2354a3c, auditoria histórica 4c70d69 e reprodução documental dos motores em 1 nov 2026.

3 Comandos e execução controlada

Checkout baseline

```
git status --short
node --version
node --test tests/*.test.mjs
git status --short
```

O glob executa a suíte ampla e inclui contratos históricos; no baseline documentado termina com falhas. Não é um comando de aprovação de release. Leia importações e efeitos dos arquivos antes de executar scripts avulsos: alguns geram relatórios ou reescrevem catálogos.

Foco em motores atuais

```
node --test tests/broto-engine.test.mjs \
tests/cerco-engine.test.mjs \
tests/sintonia-engine.test.mjs \
tests/repasse-engine.test.mjs \
tests/orbita-engine.test.mjs
```

A lista é um ponto de partida, não toda a regressão. Acrescente os testes do contrato tocado: mapas, migração, serviço, interação e preservação. O formato *.test.mjs não abrange todos os testes históricos do projeto, pois há scripts com outros nomes.

Cópia local de preparação para GitHub

```
node scripts/test.mjs --list
node scripts/test.mjs
node scripts/test.mjs --historical
node scripts/test.mjs --deferred
node scripts/test.mjs --all
```

Esses comandos pertencem à revisão local f0e7fcf, não ao baseline. O manifesto tests/test-manifest.json e docs/test-triage.md organizam a seleção. A categoria --deferred está vazia e termina sem subprocesso. Só use o runner no checkout que contém esses arquivos; a CI preparada ainda não foi executada remotamente.

Execução segura

• Use dados sintéticos e cópia de trabalho para testes de armazenamento e migração; não limpe saves reais para obter um ambiente verde.

• Registre commit, versão Node, comando, saída, duração, contagem e diff após rodar. Um log antigo não comprova uma alteração posterior.

• Não rode scripts build/record indiscriminadamente. Prefira testes de leitura quando a tarefa é auditoria.

Fontes desta seção: Inventário tests/; logs do baseline; scripts e manifesto da preparação local informados pela equipe de revisão.

4 Camadas e alcance dos testes

Camada	Exemplos	Falhas que alcança
Unidade de regra	Movimento ilegal, parceiro extra, momento não nulo	Erro determinístico do motor
Replay de catálogo	640 referências ativas	Solução quebrada ou dado inválido
Busca e certificados	Unicidade, causalidade e alternativas	Afirmação matemática não sustentada
Serviço	Worker, timeout, cancelamento e reserva	Resposta antiga, travamento ou resultado não certificado
Estado e migração	Save antigo, corrupção, identidade	Perda, transferência ou duplicação de progresso
DOM simulado	Controles, seletores e eventos	Fluxo e marcação do modelo de teste
Navegador real	Toque, foco, rolagem, tamanhos e SVG	Problema de integração/renderização
Pessoas jogando	Compreensão, ritmo e descoberta	Problema de uso ou curva editorial

Teste independente não é só outra chamada

Uma auditoria que chama o mesmo motor de produção comprova consistência de uso. Uma segunda implementação de regra pode reduzir risco de erro compartilhado, desde que realmente refaça a lógica. Compare resultados e documente diferenças, incluindo condições de contorno e tamanho do espaço.

Quando um mock falha

Confira se o navegador oferece a API que o mock omitiu. Se sim, conserte o harness e mantenha a asserção funcional. Não altere a UI para acomodar um mock frágil sem motivo de compatibilidade real. Se o mock aceita uma situação que o DOM real rejeita, a suíte verde pode esconder o defeito inverso.

Evidência visual nesta entrega

Os PDFs foram renderizados e inspecionados como documentos. A revisão local também registrou um smoke em navegador nativo: 231 verificações em 33 cenários, cobrindo 11 jogos em três larguras. Esse teste de fluxo básico não percorre as 640 fases nem certifica acessibilidade; seu alcance está na seção 11.

Fontes desta seção: Estrutura dos testes do projeto; docs/sintonia-goals-v5.md; relatórios independentes por campanha.

5 Matriz mínima de regressão

Cenário	Oráculo ou resultado esperado
Abrir categoria e jogo	Somente a tela selecionada fica ativa; retorno do navegador coerente
Primeira e última fase	80 itens solo; IDs corretos; nenhuma referência a catálogo retirado
Ação válida e inválida	Motor aplica ou rejeita sem mutação parcial
Desfazer e refazer	Estado anterior íntegro; ramo futuro tratado corretamente
Reiniciar	Mesma definição; progresso permanente não é apagado
Vencer duas vezes	Conclusão contada uma vez por tentativa/identidade
Recarregar após vitória	Conclusão preservada e sem nova premiação
Sair durante busca	Resposta antiga não altera outra tela
Falha de storage	Jogo continua quando previsto; aviso honesto de não gravação
JSON inválido ou cancelado	Partida anterior fica intacta
Modo livre	Seed/opções/definição retomam o desafio esperado
Versus	Turno, início, reservas e resultado correspondem ao replay

Diferenças que precisam de teste explícito

PACTO, SINTONIA, REPASSE e ÓRBITA detectam vitória automaticamente. Reflexos, Corte, ELO e PRUMO mantêm confirmação; PACTO ainda permite confirmar uma tentativa para avaliação. DESVIO confirma o lance; BROTO e CERCO têm seus próprios fluxos. Teste o contrato real, não uma generalização extraída de documentação antiga.

Nos duelos, rejeite clique atrasado de outro turno. Em BROTO, confirme resolução simultânea dos receptores e alternância do início na segunda rodada. Em CERCO, verifique a equipe inteira quando uma peça fica isolada. Em DESVIO, casas consumidas restringem disponibilidade, não bloqueiam um trajeto físico imaginário.

Variações de apresentação

A revisão manual recomendada inclui 320, 390 e 1366 px, teclado, toque, nomes longos, regras abertas, resultado e movimento reduzido. O smoke local cobriu o fluxo básico nas três larguras, não todas essas combinações nem toda a campanha.

Fontes desta seção: Motores e testes de integração; tests/automatic-completion-no-confirm.test.mjs e módulos de histórico.

6 Saves migrações e regressão

Monte fixtures por identidade

Uma fixture deve conter versão, definição ou identidade, estado parcial, ações, desfazer/refazer, conclusão e assistência quando aplicáveis. Crie casos para fase preservada, fase alterada com novo ID, fase retirada, registro malformado e armazenamento indisponível. Não use a posição da seleção como chave de equivalência.

Caso	Critério de aceitação
Definição preservada	Retoma estado compatível sem perder conclusão
Definição alterada	Nova identidade não herda vitória antiga por número
Jogo retirado	Registro antigo não é apagado só porque saiu do menu
Save corrompido	É recusado ou arquivado sem bloquear o restante
Ramo de refazer inválido	Não entra como futuro confiável
Metadado apenas editorial	Não reinicia sem motivo uma definição fisicamente igual
Falha de gravação	Não comunica persistência garantida
Duas leituras concorrentes	Não regride assistência ou contadores já conhecidos

Exemplo SINTONIA v5

A separação de destinos, placas e interruptores alterou nove fases: 16, 31, 36, 45, 57, 58, 60, 74 e 79. Essas definições têm IDs novos; as outras 71 permanecem iguais à revisão anterior. O teste deve provar os dois lados: impedir conclusão herdada nas nove e preservar as 71, não apenas contar que existem 80 entradas.

Duelos e replay

BROTO reconstrói as duas rodadas e compara pontuações, marcas e futuro. CERCO reproduz lances e confere barreiras, jogador, turno e vencedor. Um objeto salvo com pontos editados deve ser recusado mesmo que o desenho pareça plausível. A comparação exata é parte do contrato, não excesso de rigor que se pode retirar para aceitar fixtures antigas.

Antes de mudar o esquema, defina política de migração e backup. Em testes, retenha os bytes anteriores para comparar preservação. Na produção, não ofereça limpeza de armazenamento como primeiro tratamento de um problema não diagnosticado.

Inclua explicitamente o teste **normalizar duas vezes produz o mesmo arquivo**. A duplicação de referências históricas de SINTONIA encontrada neste baseline mostra por que uma migração que abre corretamente na primeira vez ainda pode falhar na segunda.

Fontes desta seção: dist/broto/engine.mjs, cerco/engine.mjs, *-progress.mjs; docs/sintonia-goals-v5.md.

7 Assíncrono geração e desempenho

Casos obrigatórios de concorrência

• Iniciar geração e sair do jogo antes da resposta. Reabrir outra fase. A tarefa antiga não deve pintar a nova tela.

• Pedir análise duas vezes com estados diferentes. Só o pedido atual pode publicar a resposta no canal correspondente.

• Provocar erro de worker, mensagem inválida e timeout. A reserva ou análise limitada precisa ser certificada e respeitar as opções.

• Cancelar enquanto o caminho de contingência está ativo. O abort deve valer também fora do worker.

Metas de desempenho sem promessas falsas

Orçamentos de estados e tempo protegem a interface. Medir em uma máquina não garante a mesma latência em aparelho modesto. Registre custo de primeira abertura, retomada, geração e análise separadamente. A ausência de backend não implica ausência de custo: catálogos legados grandes podem entrar por importações transitivas.

ÓRBITA é carregada sob demanda, mas uma cadeia de importação do modo livre alcança catálogo legado volumoso. Isso é motivo para medir e planejar extração, não para apagar dados de compatibilidade sem testar saves. Dados grandes não devem ser confundidos com código de função excessivo.

Quando a geração repete

Confira seed, versão, opções e histórico recente. Um conjunto finito pode esgotar candidatos e permitir reutilização fora da janela de recência. Determine se a repetição viola o contrato atual ou um teste antigo. Não prometa unicidade infinita para corrigir a expectativa.

Sinal	Investigação prioritária
UI travada	Busca síncrona, orçamento e trabalho de render
Resultado aparece após sair	Serial/revisão, cancelamento e timers
Reserva incompatível	Normalização de opções e certificação
Seed não reproduz	Versão, definição salva e parâmetros
Vitória duplicada	Idempotência de registro após replay/redo

Fontes desta seção: dist/repasse/service.mjs, cerco/service.mjs, orbita/route.mjs, orbita/endless-v2.mjs e geradores dedicados.

8 Importação entradas e limites

Teste o limite e seus vizinhos. Um arquivo com 200.000 bytes não é o mesmo que 200.000 caracteres: caracteres acentuados podem usar mais bytes. Uma entrada rejeitada deve conservar a partida, inclusive após uma busca iniciada e cancelada.

Categoria	Casos
Sintaxe	JSON truncado, vírgula final, BOM, objeto versus array
Forma	game errado, schema desconhecido, campos extras conforme contrato
Recursos	Tamanho, profundidade, nós, array, texto e chaves máximos
Dados hostis	__proto__, constructor, prototype e números inválidos
Referências	IDs repetidos, parceiro de portal inválido, índice fora da grade
Regra	Pista contraditória, apoio compartilhado, polígono autointersectante
Busca	Zero soluções, mais de uma e orçamento inconclusivo
Transação	Sair, editar texto ou cancelar antes de concluir validação
Identidade	Mesmo conteúdo com nome novo; regras novas com nome antigo
Progresso	Campos de vitória/assistência importados não contam campanha

Assimetria de limites é intencional

O importador ELO compartilhado limita grades a 7×7; PRUMO a 6×6. Campanhas podem conter grades maiores porque foram curadas e certificadas offline. Exportar uma fase maior não prova que o mesmo importador deva aceitá-la. Documente essa diferença para evitar um falso bug de ida e volta.

Conservador não significa incorreto

Curvas de Corte com lascas quase degeneradas podem ser recusadas por custo de geometria e renderização. Uma fase logicamente válida pode exceder o orçamento de busca. A mensagem deve separar formato inválido, contradição e verificação inconclusiva. Não reescreva o erro como “sem solução” quando não houve prova.

As rotas de importação dos jogos dedicados precisam de seus próprios casos. Em SINTONIA v5, há um guard para impedir sobreposição de destinos com placas ou interruptores; um teste de esquema genérico não substitui esse contrato semântico.

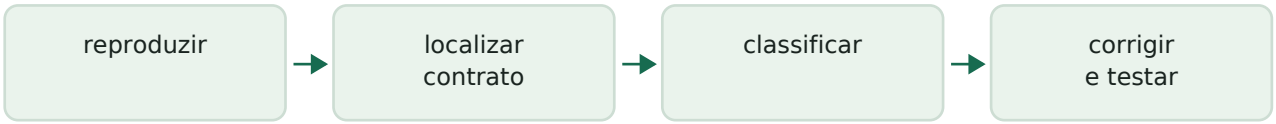
Fontes desta seção: dist/level-json-common.mjs, collection-json.mjs, original-json.mjs, sintonia/active-rules.mjs e IMPORTAR-NIVEIS-JSON.md.

9 Manutenção e triagem

Ficha mínima de defeito

Registre jogo, modo, fase/ID ou mapa, commit, ambiente, passos, esperado, observado e evidência. Inclua se havia uma sessão antiga e se a falha ocorre em perfil sintético limpo. Não inclua credenciais, caminhos privados ou conteúdo pessoal em relatórios destinados a publicação.

Classificação de uma falha



Classe	Ação correta
Regressão de regra	Corrigir produção e criar caso mínimo
Contrato superado	Atualizar teste ativo e arquivar a expectativa antiga
Mock incompleto	Consertar harness mantendo intenção funcional
Evidência histórica	Rotular e separar, sem falsificar o resultado original
Dependência de ambiente	Documentar requisito e verificar alternativa segura
Inconclusiva	Manter aberta com pergunta e evidência faltante

Refatorar sem reescrever regras

Faça mudanças pequenas: formatar, extrair função, reduzir duplicação e remover ciclo com regressão. Compare exports, dados de campanha, hashes quando pertinentes e resultados de replay. Um diff enorme de formatação junto de mudança matemática dificulta revisão; prefira separar etapas.

Testes históricos de preservação ligados a commits específicos não devem ser reescritos para fingir que sempre passaram. Na suíte ativa, adote contratos de comportamento duradouros; guarde snapshots de entrega com contexto e data. Um clone raso pode não conter o commit exigido por git show, portanto classifique essa dependência antes de criar CI.

Direitos e distribuição

Código visível e aviso de copyright não substituem licença clara. Antes de uma distribuição pública, verifique proveniência dos assets e se relatórios contêm caminhos internos. Documentação técnica não autoriza escolher uma licença, publicar repositório ou enviar um comunicado.

Fontes desta seção: Auditorias e preparação local de testes; inventário de recursos e proveniência da coleção.

10 Critérios de aceitação

Portão	Evidência para encerrar
Escopo	Mudança coincide com pedido e arquivos esperados
Regra e conteúdo	Casos tocados e replays atuais aprovados
Estado	Sessões antigas e novas preservadas conforme política
Assíncrono	Cancelamento, timeout e contingência corretos
UI	Fluxos automatizados e revisão interativa pertinente
Documentação	Contagens, comandos e limites correspondem ao checkout
Distribuição	Assets, privacidade e autorização revisados
Entrega	Commit, resultados e pendências explícitos

Não bloquear pela métrica errada

Classifique testes históricos com motivo e evidência. Não ignore uma falha ativa de regra para obter um número verde; decida por contrato e risco, não só pela porcentagem.

Estado documental desta edição

A suíte global do baseline permanece registrada com 180 falhas. A reprodução de 640 soluções passou. O fechamento local f0e7fcf passou 536/536 testes atuais; o diagnóstico completo registrou 843 testes, 670 aprovados e 173 falhas, todas em contratos históricos identificados. Nenhum arquivo misto ficou sem classificação. Os denominadores mudaram: não é uma redução direta de 180 para 173 bugs.

A correção local de SINTONIA mantém 482 referências históricas estáveis após normalizações repetidas, com 18 definições antigas idênticas retomáveis. Sete casos novos protegem aliases, payloads diferentes e colisões; o conjunto focado de migração passou 18 testes. Nenhuma dessas alterações foi publicada no Site ou enviada ao GitHub. A correção introduzida em 2dea81c integra o fechamento f0e7fcf, não o Site publicado.

O smoke nativo local e a inspeção visual destes PDFs são atividades distintas. Nenhum deles substitui todas as camadas de teste; veja na seção 11 os limites do fechamento.

Fontes de referência

Fontes: dist/, tests/ e dist/IMPORTAR-NIVEIS-JSON.md no baseline. No fechamento local f0e7fcf, consulte docs/test-triage.md, docs/test-case-decisions.md e tests/test-manifest.json. Caminhos relativos à raiz do checkout.

Na entrega, discrimine executado, aprovado, falho e não observado. Evite promessas como “sem bugs” ou “totalmente testado”.

11 Fechamento da revisão local

Atualização de 1 nov 2026. Commit local **f0e7fcf150606fc785b0b932c282c8fb5b8f0419**, em cópia separada e sem remotes. O Site permanece no baseline 2354a3c. Nada foi enviado ao GitHub ou implantado.

Verificação final	Resultado e alcance
Contratos atuais	536 testes aprovados em 108 arquivos; zero falhas
Classificação	87 arquivos históricos preservados; zero mistos/deferred
Diagnóstico completo	843 testes: 670 aprovados e 173 falhas históricas identificadas
Sintaxe e preservação	153 módulos válidos; motores, campanhas, imagens, fontes, estilos e regras preservados
Migração SINTONIA	482 referências históricas estáveis; aliases distintos e colisões preservados

O que mudou na triagem

Os casos ainda vigentes foram extraídos de provas pontuais de releases e requisitos substituídos. O manifesto cobre os 195 arquivos exatamente uma vez e aponta os testes substitutos. Zero pendências de classificação não significa que toda falha histórica foi uma correção de produto ou que não possam existir defeitos não testados.

Navegador e limites

Foram 231 verificações em 33 cenários: os 11 jogos nas larguras 1366, 390 e 320. Entrada, presença do tabuleiro, overflow horizontal, cancelamento de reset quando disponível e retorno passaram sem erros; saves foram restaurados. As telas compactas usaram iframe real, não dispositivo físico. Não é revisão visual de todas as 640 fases nem teste completo de acessibilidade.

Segurança e publicação

A auditoria de exportação no snapshot cf7346f não encontrou correspondências nos padrões de credenciais examinados em texto rastreado, histórico textual alcançável e JSONs gzip atuais. Não houve OCR dos pixels nem análise profunda de todos os binários; resultado zero não garante ausência de segredos. A prosa de três documentos foi normalizada, preservando certificados e histórico.

Pendências antes de tornar público: escolher licença do código, confirmar direitos dos ativos e decidir o conteúdo do export. Um snapshot sem histórico não reproduz toda a QA baseada em Git. A CI foi preparada, mas ainda precisa de execução remota após autorização. A revisão local não autoriza envio ou implantação.

Fontes desta seção: Cópia f0e7fcf: docs/local-review-summary.md, docs/test-triage.md, docs/test-case-decisions.md, docs/export-review.md e docs/review-evidence/native-route-smoke.json.